

1. GitHub

- Основная ветка `main` содержит каркас программы

Блок-схемы

- **Блок-схема 1** — общая архитектура всего проекта (как главный скрипт вызывает модули, меню, потоки данных). Оформляется в `README.md`.
- **Блок-схема 2** — детальная схема алгоритма работы вашего конкретного модуля. Добавляется в папку `docs/` в формате PNG или встроена в Markdown-файл рядом с модулем.

2. Код модуля

- Каждый модуль — отдельный `.py` файл в папке `modules/`.
- Обязательно наличие функции с документ-строкой, описывающей параметры и возвращаемое значение.
- При ошибочных входных данных модуль должен выбрасывать понятное исключение или возвращать код ошибки.

Техническое задание

Вам предоставлен базовый скрипт на Python — каркас приложения для обработки текста. Ваша группа делится на подгруппы (по 1–2 человека). Каждая подгруппа разрабатывает **отдельный модуль**, реализующий определённую функцию обработки текста.

Список модулей для разработки (каждая подгруппа берёт одну роль):

- **Роль 1 — Анализатор частоты слов.** Подсчитывает, сколько раз каждое слово встречается в тексте, выводит топ-N.
- **Роль 2 — Шифратор Цезаря.** Выполняет шифрование/дешифрование текста сдвигом по алфавиту (с сохранением регистра и знаков препинания).
- **Роль 3 — Форматировщик в Upper/Lower/Title.** Преобразует текст по выбору пользователя в верхний, нижний регистр или начинающийся с заглавных букв.
- **Роль 4 — Генератор облака слов.** Сохраняет облако слов в виде PNG-изображения (использовать библиотеку `wordcloud`).
- **Роль 5 — Измеритель сложности текста.** Рассчитывает индекс удобочитаемости Флеша (или аналог) для русского/английского текста.
- **Роль 6 — Поиск и замена.** Ищет в тексте заданную подстроку (с учётом/без учёта регистра) и заменяет её на другую.

- **Роль 7 — Извлечение email и телефонных номеров.** Находит и возвращает список всех email-адресов и телефонных номеров российского формата.
- **Роль 8 — Конвертер в транслит.** Переводит кириллический текст в латиницу по стандарту ISO 9.
- **Роль 9 — Анализатор тональности.** Простейший анализ тональности на основе словарей позитивных/негативных слов.
- **Роль 10 — Генератор N-грамм.** Разбивает текст на N-граммы (последовательности из N слов) и выводит их количество.

Пример готового модуля «Анализатор частоты слов» (Роль 1)

```

import re
from collections import Counter

def analyze(text: str, top_n: int = 10) -> str:
    """
    Анализирует частоту слов в тексте.

    Параметры:
    | text (str): исходный текст.
    | top_n (int): количество наиболее частых слов для вывода.

    Возвращает:
    | str: отформатированная строка с топ-N словами и их частотой.

    Исключения:
    | ValueError: если текст пуст или top_n < 1.
    """
    if not text or not text.strip():
        raise ValueError("Текст не должен быть пустым.")
    if top_n < 1:
        raise ValueError("top_n должен быть >= 1")

    # Приведение к нижнему регистру и извлечение слов
    words = re.findall(r'\b\w+\b', text.lower())
    counter = Counter(words)

    if not counter:
        return "Слов не найдено."

    most_common = counter.most_common(top_n)
    lines = [f"{word}: {count}" for word, count in most_common]
    return "\n".join(lines)

```

Роль 2 — Шифратор Цезаря

Файл: modules/caesar_cipher.py

Функция: cipher(text: str, shift: int, decrypt: bool = False) -> str

Алфавит: английский + русский. Сдвиг — целое число. Если decrypt=True, сдвиг в обратную сторону. Знаки препинания и пробелы не меняются.

Роль 3 — Форматировщик регистров

Файл: `modules/case_formatter.py`

Функция: `format_case(text: str, mode: str) -> str`

`mode` — одно из: 'upper', 'lower', 'title'. При некорректном `mode` выбрасывается `ValueError`.

Роль 4 — Генератор облака слов

Файл: `modules/wordcloud_gen.py`

Функция: `generate(text: str, output_path: str = "wordcloud.png") -> str`

Сохраняет изображение, возвращает путь к нему. Не забудьте добавить `wordcloud` в `requirements.txt`.

Роль 5 — Измеритель сложности текста

Файл: `modules/readability.py`

Функция: `flesch_index(text: str) -> str`

Рассчитывает индекс удобочитаемости Флеша для русского текста. Формула: $206.835 - 1.3 * (\text{слов} / \text{предложений}) - 60.1 * (\text{слогов} / \text{слов})$. Вернуть строку "Индекс: X.XX (Уровень: ...)".

Роль 6 — Поиск и замена

Файл: `modules/find_replace.py`

Функция: `replace(text: str, find: str, replace_with: str, case_sensitive: bool = True) -> str`

Возвращает изменённый текст.

Роль 7 — Извлечение email и телефонов

Файл: `modules/contact_extractor.py`

Функция: `extract(text: str) -> str`

Ищет email (примерный regex) и российские номера (+7, 8, с пробелами и скобками). Возвращает строку с двумя списками.

Роль 8 — Транслитерация

Файл: `modules/translit.py`

Функция: `to_latin(text: str) -> str`

Использовать словарь соответствия букв по ISO 9. Буквы не из кириллицы оставлять без изменений.

Роль 9 — Анализатор тональности

Файл: modules/sentiment.py

Функция: analyze(text: str) -> str

Внутри модуля определить списки позитивных и негативных слов (не менее 10-15 каждого). Возвращает "Позитивный", "Негативный" или "Нейтральный".

Роль 10 — Генератор N-грамм

Файл: modules/ngrams.py

Функция: generate(text: str, n: int = 2) -> str

Разбивает текст на N-граммы слов, считает их частоту, возвращает строку с топ-15.